STEP 1 — WINDOWS LOG INGESTION (FULL EXPLANATION)

Full Code (for reference)

```
import win32evtlog

def collect_windows_events(limit=50):
  server = 'localhost'
  logtype = 'Security'

  hand = win32evtlog.OpenEventLog(server, logtype)
  flags = win32evtlog.EVENTLOG_BACKWARDS_READ |
win32evtlog.EVENTLOG_SEQUENTIAL_READ

  events = []

  while len(events) < limit:
    records = win32evtlog.ReadEventLog(hand, flags, 0)
    if not records:
      break

    for event in records:
      events.append({
        "event_id": event.EventID,
        "timestamp": str(event.TimeGenerated),
        "message": event.StringInserts
      })

      if len(events) >= limit:
        break

  return events
```

1️⃣ IMPORT

```
import win32evtlog
```

👉 This is from **pywin32**, a library that lets Python interact with Windows internals.

✔ Access:

- Event Viewer logs
- System APIs

👉 This is how you pull logs directly from Windows (like a SIEM agent)

---

2 FUNCTION DEFINITION

def collect_windows_events(limit=50):

👉 Creates a function that:

Collects up to 50 Windows log events

✔ You can change `limit` to:

- 100 (more logs)
- 1000 (bulk ingestion)

---

3 TARGET SYSTEM + LOG TYPE

server = 'localhost'
logtype = 'Security'

`server`

'localhost' = your own machine

👉 Could also be:

- Remote server (in enterprise SOC)

`logtype`

'Security' log = authentication + security events

👉 Includes:

- Logins (4624, 4625)

- Process creation (4688)
- File access

---

4 OPEN EVENT LOG

hand = win32evtlog.OpenEventLog(server, logtype)

👉 Think of this like:

Opening Event Viewer programmatically

✔ Returns a **handle (connection)** to the log

---

5 READ MODE FLAGS

flags = win32evtlog.EVENTLOG_BACKWARDS_READ |
win32evtlog.EVENTLOG_SEQUENTIAL_READ

What this means:

✔ `EVENTLOG_BACKWARDS_READ`

Start from newest logs → go backwards

👉 Important for:

- Real-time detection
- Recent attacks

✔ `EVENTLOG_SEQUENTIAL_READ`

Read logs in order (one by one)

Combined:

Read newest logs first, one-by-one

---

6 STORAGE

events = []

This will store all collected logs:

Python list of event dictionaries

---

7 MAIN LOOP (CORE LOGIC)

while len(events) < limit:

👉 Keep collecting logs until:

You reach the limit (e.g. 50 events)

---

8 READ LOG RECORDS

records = win32evtlog.ReadEventLog(hand, flags, 0)

👉 This is the **actual log retrieval**

✔ Returns:

A batch of Windows event objects

If no logs:

```
if not records:
  break
```

👉 Stops when:

- No more logs available

---

9 PROCESS EACH EVENT

for event in records:

👉 Loop through each Windows log entry

EXTRACT IMPORTANT FIELDS

```
events.append({
  "event_id": event.EventID,
  "timestamp": str(event.TimeGenerated),
  "message": event.StringInserts
})
```

`event.EventID`

The type of event (e.g. 4625)

This is your **detection trigger**

`event.TimeGenerated`

When the event happened

Used for:

- Timeline reconstruction

`event.StringInserts`

Extra details (user, process, IP, etc.)

Example:

["admin", "192.168.1.5"]

---

1 LIMIT CHECK

```
if len(events) >= limit:
  break
```

Stops once enough logs are collected

---

1 2 RETURN DATA

return events

👉 Output:

```
[
 {
   "event_id": 4625,
   "timestamp": "2026-01-01",
   "message": ["admin", "192.168.1.5"]
 }
]
```

WHAT THIS CODE REALLY DOES

In simple terms:

Connect → Read Windows logs → Extract key fields → Return structured data

---

In SOC / SIEM terms:

This is a log ingestion agent

WHY THIS IS IMPORTANT

Without this:

No data → No detection → No investigation

With this:

✔ You collect raw security events
✔ You feed your detection engine
✔ You enable correlation (next steps)

HOW IT CONNECTS TO YOUR PIPELINE

Windows Logs
↓
windows_ingest.py ← YOU ARE HERE
↓
Normalizer
↓
MITRE Mapping
↓
Detection Engine
↓
SIEM Dashboard

IMPORTANT LIMITATIONS

❗ This code does NOT yet:

❌ Parse IPs cleanly
❌ Extract usernames properly
❌ Handle large-scale ingestion
❌ Work on macOS/Linux

It's a **foundation**, not production-ready

HOW TO IMPROVE IT (NEXT LEVEL)

✔ Extracting real fields

User

Source IP
Process name
PPID

✔ Filter only important events

if event.EventID in [4624, 4625, 4688]:

✔ Stream logs continuously

Instead of batch:

Real-time ingestion (like Splunk/Elastic)

FINAL UNDERSTANDING

This script is:

Your SIEM's "eyes" into Windows

It gives:

Event ID → WHAT happened
Timestamp → WHEN
Message → DETAILS